འབྲུག་རྒྱལ་འཛིན་གཙུག་ལག་སློབ་

College of Science and Technology
Rinchending: Bhutan

DBS101
Database Systems Fundamentals
(SS2026)

Practical 1 Report

Submitted By; Ashis Rai
Name: Ashis Rai
Student No.: 02240334
Programme: BE 1SWE
Date: 21/03/2026

RUB Wheel of Academic Law: Academic Dishonesty

Section H2 of the Royal University of Bhutan's *Wheel of Academic Law* provides the following definition of academic dishonesty:

Academic dishonesty may be defined as any attempt by a student to gain an unfair advantage in any assessment. It may be demonstrated by one of the following:

1. **Collusion:** the representation of a piece of unauthorised group work as the work of a single candidate.
2. **Commissioning:** submitting an assignment done by another person as the student's own work.
3. **Duplication**: the inclusion in coursework of material identical or substantially similar to material which has already been submitted for any other assessment within the University.
4. **False declaration**: making a false declaration to receive special consideration by an Examination Board or to obtain extensions to deadlines or exemption from work.
5. **Falsification of data**: presentation of data in laboratory reports, projects, etc., based on work purported to have been carried out by the student, which has been invented, altered or copied by the student.
6. **Plagiarism**: the unacknowledged use of another's work as if it were one's own.

Examples are:

- verbatim copying of another's work without acknowledgement.
- paraphrasing of another's work by simply changing a few words or altering the order of presentation, without acknowledgement.
- ideas or intellectual data in any form presented as one's own without acknowledging the source(s).
- making significant use of unattributed digital images such as graphs, tables, photographs, etc., taken from test books, articles, films, plays, handouts, the internet, or any other source, whether published or unpublished.
- submission of a piece of work which has previously been assessed for a different award or module or at a different institution as if it were new work.
- use of any material without prior permission of copyright from the appropriate authority or owner of the materials used".

Aim

The aim of this lab is to implement a relational database for a college management system by creating tables, inserting data, and executing various SQL queries to manipulate and retrieve information which will be done by using SQL commands such as DDL, DML and SQL Queries in this lab.

Theory

Relational Database Management System (RDBMS) is a program that allows users to perform certain operation such as create, delete, manage and interact with relational databases. RDBMS organizes data in the form of table with rows and columns.
SQL commands are used carry out operation including creating, deleting and managing data. The following SQL commands are used in this lab:

1. Data Definition Language (DDL): Used to define the database structure
   - CREATE DATABASE: Creates a database.
   - CREATE TABLE: Creates a table inside the database.
   - FOREIGN KEY: Links between two table.
   - PRIMARY KEY: Defines that entries should be neither null or duplicate.
   - NOT NULL: Ensure that a column cannot store NULL (empty) value.

2. Data Manipulation Language (DML): Used to manage the data stored in the tables.
   - INSERT INTO: Adds records into table.
   - SELECT: Used to retrieve data from table.
   - UPDATE: Update existing data.
   - DELETE: Deletes existing data.

3.SQL Queries: Used to filter, group, sort and control the table by SELECT statement.
   - WHERE: Filters records and return values that satisfy specific conditions.
   - JOIN: Combines columns from two or more tables based on a related column
   - GROUP BY: Groups rows that have same value in specified columns.

Implementation Steps

Task 1: Create Database and table

Step 1: Create Database: college_db

SQL command:

```
mysql> CREATE DATABASE college_db
    -> ;
Query OK, 1 row affected (0.02 sec)
```

Output:

```
mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| college_db         |
| information_schema |
| mysql              |
| performance_schema |
| swe                |
| sys                |
+--------------------+
6 rows in set (0.00 sec)
```

Step 2: Select the database using USE statement

SQL command:

```
mysql> USE college_db;
Database changed
mysql> |
```

Step 3: Create department table with the constraints:

1.dept_id
2.dept_name
3.hod_name

SQL command:

```
mysql> CREATE TABLE department(
    -> dept_id INT PRIMARY KEY,
    -> dept_name VARCHAR(50) NOT NULL,
    -> hod_name VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.04 sec)
```

Step 4: Create student table with the constraints:

1.student_id
2.student_name
3.age
4.city
5.Dept_id

SQL command:

```
mysql> CREATE TABLE student(
    -> student_id INT PRIMARY KEY,
    -> student_name VARCHAR(50) NOT NULL,
    -> age INT,
    -> city VARCHAR(50),
    -> dept_id INT,
    -> FOREIGN KEY (dept_id) REFERENCES department(dept_id)
    -> );
Query OK, 0 rows affected (0.05 sec)
```

Step 5: Verify Table

```
mysql> SHOW TABLES;
+----------------------+
| Tables_in_college_db |
+----------------------+
| department           |
| student              |
+----------------------+
2 rows in set (0.02 sec)
```

Task 2: Insert Records into Tables

Step 1: Insert the following records into the department table:

dept_id	dept_name	hod_name
1	Information Technology	Sonam Choden
2	Business Management	Pema Dorji
3	Civil Engineering	Kinley Wangmo

SQL command:

```
mysql> INSERT INTO department
    -> VALUES
    -> (1, 'Information Technology','Sonam Choden'),
    -> (2,'Business Management','Pema Dorji'),
    -> (3,'Civil Engineering','Kinley Wangmo');
Query OK, 3 rows affected (0.03 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

Step 2: Insert the following records into the student table:

student_id	student_name	age	city	dept_id
101	Tashi Dorji	20	Thimphu	1
102	Pema Lham	21	Paro	2
103	Sonam Wangchuk	19	Punakha	1
104	Dechen Yangzom	22	Gelephu	3
105	Karma Tshering	20	Samdrup Jongkhar	2

SQL command:

```
mysql> INSERT INTO student
    -> VALUES
    -> (101,'Tashi Dorji',20,'Thimphu',1),
    -> (102,'Pema Lham',21,'Paro',2),
    -> (103,'Sonam Wangchuk',19,'Punakha',1),
    -> (104,'Dechen Yangzom',22,'Gelephu',3),
    -> (105,'Karma Tshering',20,'Samdrup Jongkhar',2);
Query OK, 5 rows affected (0.02 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Step 3: Display all records from both tables

```
mysql> SELECT * FROM department;
+---------+-----------------------+---------------+
| dept_id | dept_name             | hod_name      |
+---------+-----------------------+---------------+
|       1 | Information Technology | Sonam Choden  |
|       2 | Business Management    | Pema Dorji    |
|       3 | Civil Engineering      | Kinley Wangmo |
+---------+-----------------------+---------------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM student;
+------------+----------------+------+------------------+---------+
| student_id | student_name   | age  | city             | dept_id |
+------------+----------------+------+------------------+---------+
|        101 | Tashi Dorji    |   20 | Thimphu          |       1 |
|        102 | Pema Lham      |   21 | Paro             |       2 |
|        103 | Sonam Wangchuk |   19 | Punakha          |       1 |
|        104 | Dechen Yangzom |   22 | Gelephu          |       3 |
|        105 | Karma Tshering |   20 | Samdrup Jongkhar |       2 |
+------------+----------------+------+------------------+---------+
5 rows in set (0.00 sec)
```

Task 3: Show student names with their department names

SQL command:

```
mysql> SELECT s.student_name,d.dept_name
    -> FROM student s
    -> INNER JOIN department d ON s.dept_id =
    -> d.dept_id;
```

Output:

```
+----------------+-----------------------+
| student_name   | dept_name             |
+----------------+-----------------------+
| Tashi Dorji    | Information Technology |
| Sonam Wangchuk | Information Technology |
| Pema Lham      | Business Management    |
| Karma Tshering | Business Management    |
| Dechen Yangzom | Civil Engineering      |
+----------------+-----------------------+
5 rows in set (0.00 sec)
```

Task 4: Display students whose age is greater than 20

SQL command:

```
mysql> SELECT * FROM student
    -> WHERE age>20;
```

Output:

```
+------------+----------------+------+---------+---------+
| student_id | student_name   | age  | city    | dept_id |
+------------+----------------+------+---------+---------+
|        102 | Pema Lham      |   21 | Paro    |       2 |
|        104 | Dechen Yangzom |   22 | Gelephu |       3 |
+------------+----------------+------+---------+---------+
2 rows in set (0.00 sec)
```

Task 5: Display students who belong to the Information Technology department

SQL command:

```
mysql> SELECT s.*
    -> FROM student s
    -> INNER JOIN department d ON s.dept_id =
    -> d.dept_id
    -> WHERE d.dept_name = 'Information Technology';
```

Output:

```
+------------+----------------+------+---------+---------+
| student_id | student_name   | age  | city    | dept_id |
+------------+----------------+------+---------+---------+
|        101 | Tashi Dorji    |   20 | Thimphu |       1 |
|        103 | Sonam Wangchuk |   19 | Punakha |       1 |
+------------+----------------+------+---------+---------+
2 rows in set (0.01 sec)
```

Task 6: Update the city of Tashi Dorji to Wangdue.

SQL command:

```
mysql> UPDATE student
    -> SET city = 'Wangdue'
    -> WHERE student_name = 'Tashi Dorji';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Output:

```
mysql> SELECT * FROM student;
+------------+----------------+------+------------------+---------+
| student_id | student_name   | age  | city             | dept_id |
+------------+----------------+------+------------------+---------+
|        101 | Tashi Dorji    |   20 | Wangdue          |       1 |
|        102 | Pema Lham      |   22 | Paro             |       2 |
|        103 | Sonam Wangchuk |   19 | Punakha          |       1 |
|        104 | Dechen Yangzom |   22 | Gelephu          |       3 |
|        105 | Karma Tshering |   20 | Samdrup Jongkhar |       2 |
+------------+----------------+------+------------------+---------+
5 rows in set (0.00 sec)
```

Task 7: Increase the age of Pema Lham by 1.

SQL command:

```
mysql> UPDATE student
    -> SET age = age+1
    -> WHERE student_name = 'Pema Lham';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

Output:

```
mysql> SELECT * FROM student;
+------------+----------------+------+------------------+---------+
| student_id | student_name   | age  | city             | dept_id |
+------------+----------------+------+------------------+---------+
|        101 | Tashi Dorji    |   20 | Wangdue          |       1 |
|        102 | Pema Lham      |   22 | Paro             |       2 |
|        103 | Sonam Wangchuk |   19 | Punakha          |       1 |
|        104 | Dechen Yangzom |   22 | Gelephu          |       3 |
|        105 | Karma Tshering |   20 | Samdrup Jongkhar |       2 |
+------------+----------------+------+------------------+---------+
5 rows in set (0.00 sec)
```

Task 8: Delete the record of Karma Tshering from the student table.

SQL command:

```
mysql> DELETE FROM student
    -> WHERE student_name = 'kARMA Tshering';
Query OK, 1 row affected (0.02 sec)
```

Output:

```
mysql> SELECT * FROM student;
+------------+----------------+------+---------+---------+
| student_id | student_name   | age  | city    | dept_id |
+------------+----------------+------+---------+---------+
|        101 | Tashi Dorji    |   20 | Wangdue |       1 |
|        102 | Pema Lham      |   22 | Paro    |       2 |
|        103 | Sonam Wangchuk |   19 | Punakha |       1 |
|        104 | Dechen Yangzom |   22 | Gelephu |       3 |
+------------+----------------+------+---------+---------+
4 rows in set (0.00 sec)
```

Task 9: Display the final contents of the student table.

```
mysql> SELECT * FROM student;
+------------+----------------+------+---------+---------+
| student_id | student_name   | age  | city    | dept_id |
+------------+----------------+------+---------+---------+
|        101 | Tashi Dorji    |   20 | Wangdue |       1 |
|        102 | Pema Lham      |   22 | Paro    |       2 |
|        103 | Sonam Wangchuk |   19 | Punakha |       1 |
|        104 | Dechen Yangzom |   22 | Gelephu |       3 |
+------------+----------------+------+---------+---------+
4 rows in set (0.00 sec)
```

Task 10: Write a query to count how many students are in each department

SQL command:

```
mysql> SELECT d.dept_name,COUNT(s.student_id)
    -> AS student_count
    -> FROM department d
    -> LEFT JOIN student s ON d.dept_id = s.dept_id
    -> GROUP BY d.dept_name;
```

Output:

```
+------------------------+---------------+
| dept_name              | student_count |
+------------------------+---------------+
| Information Technology |             2 |
| Business Management    |             1 |
| Civil Engineering      |             1 |
+------------------------+---------------+
3 rows in set (0.02 sec)
```

Task 11: Write a query to display:

- student name
- city
- department name
- HOD name

SQL command:

```
mysql> SELECT s.student_name,s.city,d.dept_name,d.hod_name
    -> FROM student s
    -> INNER JOIN department d ON s.dept_id =
    -> d.dept_id;
```

Output:

```
+---------------+---------+-----------------------+---------------+
| student_name  | city    | dept_name             | hod_name      |
+---------------+---------+-----------------------+---------------+
| Tashi Dorji   | Wangdue | Information Technology | Sonam Choden  |
| Sonam Wangchuk | Punakha | Information Technology | Sonam Choden  |
| Pema Lham     | Paro    | Business Management   | Pema Dorji    |
| Dechen Yangzom | Gelephu | Civil Engineering     | Kinley Wangmo |
+---------------+---------+-----------------------+---------------+
4 rows in set (0.00 sec)
```

Task 12: Write a query to show only departments that have more than 1 student

SQL command:

```
mysql> SELECT d.dept_name,COUNT(s.student_id)
    -> AS student_count
    -> FROM department d
    -> INNER JOIN student s ON d.dept_id = s.dept_id
    -> GROUP BY d.dept_name
    -> HAVING COUNT(s.student_id)>1;
```

Output:

```
+-----------------------+---------------+
| dept_name             | student_count |
+-----------------------+---------------+
| Information Technology |             2 |
+-----------------------+---------------+
1 row in set (0.01 sec)
```

Conclusion

This lab work help me implement a relational database from scratch. I was able to create a database, define tables with primary and foreign keys to establish relationship and populate them with data. It also help me understand on how to use different SQL commands as DDL, DML and SQL queries. I learned to use SELECT statements for retrieving data, WHERE statement for filtering and the UPDATE and DELETE statements for data manipulation. The task also helped to understand JOIN operations which are crucial for combining data for multiple related tables such as linking student to their respective departments.

Challenges I faced was correctly implementing the foreign key constraint. I initially received an error because the tables were created in the wrong order; the student table could not reference the department table before it existed. I overcame this by ensuring the department table was created first. Additionally, understanding the difference between the WHERE and HAVING clauses was a key takeaway from the final task. WHERE filters individual rows before grouping, while HAVING filters groups of rows after the aggregation is performed. This exercise was invaluable for building a solid foundation in SQL and understanding how to translate real-world business questions into effective database queries.

Reference

GeeksforGeeks. (2025, July 23). *RDBMS benefits and limitations*. Retrieved March 19, 2026, from https://www.geeksforgeeks.org/dbms/rdbms-benefits-and-limitations/

GeeksforGeeks. (2025, July 12). *RDBMS full form – Relational database management systems*. Retrieved March 19, 2026, from https://www.geeksforgeeks.org/dbms/rdbms-full-form/

GeeksforGeeks. (2026, February 7). *SQL concepts and queries*. Retrieved March 19, 2026, from https://www.geeksforgeeks.org/sql/sql-concepts-and-queries/

GeeksforGeeks. (2024, December 27). *SQL USE database statement*. Retrieved March 19, 2026, from https://www.geeksforgeeks.org/sql/sql-use-database-statement/

GeeksforGeeks. (2026, March 17). *What is SQL?* Retrieved March 19, 2026, from https://www.geeksforgeeks.org/sql/what-is-sql/

W3Schools. (n.d.). *SQL introduction*. Retrieved March 21, 2026, from https://www.w3schools.com/sql/sql_intro.asp

W3Schools. (n.d.). *SQL syntax*. Retrieved March 21, 2026, from https://www.w3schools.com/SQl/sql_syntax.asp